

Technical Specification (Pet Project)

Table of Contents

1. Introduction
 2. Project Goals
 3. User Roles
 4. Functional Requirements
 5. Application Architecture
 6. Database Requirements
 7. Technical Requirements
 8. UX / UI Requirements
 9. SEO Requirements
 10. Code Quality
 11. Non-functional Requirements
 12. Acceptance Criteria
 13. Additional Tasks
-

1. Introduction

Next Library is an online library for browsing books and authors, built with **Next.js 16 (App Router)** and **MongoDB**.

The project is intended to train developers in a modern fullstack development approach using Next.js.

2. Project Goals

- Learn the App Router architecture
 - Understand the difference between Server and Client Components
 - Learn how to work with Server Actions
 - Master integration of Next.js and MongoDB
 - Understand SSR / SSG / ISR concepts
 - Learn to write production-oriented code
-

3. User Roles

Role	Description
Guest	Unauthenticated user
User	Authenticated user
Admin	System administrator

4. Functional Requirements

4.1 Public Section

4.1.1 Home Page

- Displays:
 - list of popular books
 - list of new books
 - Data is loaded via **Server Components**
 - Render type: **SSG + ISR**
-

4.1.2 Book Catalog

- Displays a list of books with:
 - title
 - author
 - rating
 - publication year
 - Supports:
 - pagination
 - genre filtering
 - sorting (by rating, by date)
 - Filters are implemented via **Client Components**
 - Main list is loaded via **Server Components**
 - Render type: **ISR**
-

4.1.3 Book Page

- Displays:
 - full book information
 - author information
 - average rating
 - list of reviews
 - Ability to add a review (authenticated users only)
 - Render type: **SSR**
 - Use `generateMetadata` for SEO
-

4.1.4 Authors Catalog

- List of authors:
 - name
 - number of books
 - Render type: **SSG**
-

4.1.5 Author Page

- Author biography
 - List of the author's books
 - Render type: **ISR**
-

4.2 Authentication and Users

4.2.1 Registration

- Registration via email and password
 - Validation:
 - email must be unique
 - password must be at least 6 characters
 - Implemented via **Server Actions**
-

4.2.2 Authentication

- Uses **NextAuth**
 - Credentials Provider
 - Session is available in Server Components
-

4.2.3 Access Control

Capability	User	Admin
Add reviews	✓	✓
Add to favorites	✓	✓
Manage books	✗	✓
Manage authors	✗	✓

4.3 User Dashboard

- View list of favorite books
 - Remove books from favorites
 - View own reviews
 - Delete reviews
-

4.4 Admin Panel

4.4.1 General Requirements

- Available only to users with the **Admin** role
 - Route protection via **Middleware**
-

4.4.2 Book Management

- CRUD operations:
 - create
 - edit
 - delete
 - Use:
 - Server Actions
 - `revalidateTag`
-

4.4.3 Author Management

- CRUD operations for authors

- Deleting an author is forbidden if the author has books
-

5. Application Architecture

```
app/
├─ (public)
├─ (auth)
├─ (dashboard)
├─ api
├─ layout.tsx
├─ loading.tsx
├─ error.tsx
└─ not-found.tsx
```

- Use route groups
- Use nested layouts
- Usage of `pages/` is forbidden

6. Database Requirements

6.1 Book

- `title` : string
 - `description` : string
 - `authorId` : ObjectId
 - `year` : number
 - `genres` : string[]
 - `rating` : number
 - `createdAt` : Date
-

6.2 Author

- `name` : string
 - `bio` : string
 - `birthDate` : Date
-

6.3 User

- `email` : string
 - `password` : string
 - `role` : USER | ADMIN
 - `favorites` : ObjectId[]
-

6.4 Review

- `bookId` : ObjectId
 - `userId` : ObjectId
 - `rating` : number (1–5)
 - `comment` : string
-

7. Technical Requirements

- Use **Next.js 16**
 - Use **TypeScript**
 - Use **MongoDB + Mongoose**
 - Use **App Router**
 - All data mutations must be done via **Server Actions**
 - Use **Middleware** to protect routes
 - API Routes are allowed only for public APIs
-

8. UX / UI Requirements

- Implement the following UI states:
 - Loading
 - Error
 - Empty state
 - Use Next.js special files:
 - `loading.tsx`
 - `error.tsx`
 - `not-found.tsx`
 - All forms must have server-side validation
 - Errors must be clear and user-friendly
-

9. SEO Requirements

- Use **static metadata**
 - Use **dynamic metadata**
 - Implement **OpenGraph metadata**
 - All pages must have correct `title` and `description`
-

10. Code Quality

- Usage of `any` is forbidden
 - All props and functions must be typed
 - Use **ESLint** and **Prettier**
 - Code must be readable and well-structured
 - Business logic duplication is forbidden
-

11. Non-functional Requirements

- First render time must not exceed 2 seconds
 - Minimize client-side JavaScript
 - Use caching and revalidation
 - No errors in the browser console
-

12. Acceptance Criteria (Definition of Done)

The project is considered complete if:

- All pages work correctly
 - MongoDB integration is implemented
 - App Router is used
 - Server Actions are implemented
 - Authentication is implemented
 - Middleware is used
 - SEO requirements are met
 - The project successfully passes `lint` and `build`
-

13. Additional Tasks (Optional)

- Implement dark mode
- Add full-text search
- Implement E2E tests (Playwright)
- Perform performance optimization